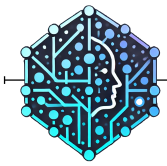


LABORATORIO DI INTELLIGENZA ARTIFICIALE APPLICATA

A.K.A. AI APPLICATIONS IN PRACTICE

5 - Advanced Concepts

Prof. Tiberio Uricchio
< tiberio.uricchio@unipi.it >

DECORATORS

Decorators

Decorators are a powerful feature in Python that allows you to modify the behavior of functions or methods without changing their source code.

A decorator is a function that takes another function as an argument, adds some functionality, and returns the modified function.

```
# Simple function
def greet():
    return "Hello!"

# Decorator function
def uppercase_decorator(func):
    def wrapper():
        original_result = func()
        modified_result = original_result.upper()
        return modified_result
    return wrapper

# Applying the decorator manually
greet = uppercase_decorator(greet)
print(greet())  # Output: HELLO!
```


Decorators syntax

A decorator can be applied by using the @ symbol.

```
# Decorator function
def uppercase_decorator(func):
    def wrapper():
        original_result = func()
        modified_result = original_result.upper()
        return modified_result
    return wrapper

# Applying the decorator using @ syntax
@uppercase_decorator
def greet():
    return "Hello!"

print(greet())  # Output: HELLO!
```

Decorators with Arguments

When the decorated function accepts arguments, the wrapper function needs to accept the same arguments.

```
def uppercase_decorator(func):
    def wrapper(*args, **kwargs):
        original_result = func(*args, **kwargs)
        return original_result.upper()
    return wrapper

@uppercase_decorator
def greet(name):
    return f"Hello, {name}!"

print(greet("Alice"))  # Output: HELLO, ALICE!
```


Preserving Function Metadata

When you apply a decorator, the original function's metadata (name, docstring, etc.) is lost. The *functools.wraps* decorator helps preserve this information.

```
import functools

def uppercase_decorator(func):
    @functools.wraps(func)  # Preserves metadata
    def wrapper(*args, **kwargs):
        original_result = func(*args, **kwargs)
        return original_result.upper()
    return wrapper

@uppercase_decorator
def greet(name):
    """Return a greeting message for the given name."""
    return f"Hello, {name}!"

print(greet.__name__)  # Output: greet (without wraps it would be wrapper)
print(greet.__doc__)   # Output: Return a greeting message for the given name.
```

Decorators with Parameters

Decorators can also accept their own parameters by adding another level of nesting.

```
def repeat(n=1):
    def decorator(func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            result = None
            for _ in range(n):
                result = func(*args, **kwargs)
            return result
        return wrapper
    return decorator

@repeat(n=3)
def say_hello():
    print("Hello!")
    return "Done"

say_hello()  # Output: Hello! Hello! Hello!
```


Real-World Examples

Decorators have many practical applications in Python:

Timing function execution

```
import time
import functools

def timer(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        start_time = time.time()
        result = func(*args, **kwargs)
        end_time = time.time()
        print(f"Function {func.__name__} took {end_time - start_time:.4f} seconds to run")
        return result
    return wrapper

@timer
def slow_function():
    time.sleep(1)
    return "Function completed"

slow_function()  # Output: Function slow_function took 1.0005 seconds to run
```

Real-World Examples

Authentication and Authorisation

```
def require_auth(func):
    @functools.wraps(func)
    def wrapper(user, *args, **kwargs):
        if not user.is_authenticated:
            return "Authentication required"
        return func(user, *args, **kwargs)
    return wrapper

@require_auth
def view_profile(user):
    return f"Welcome to your profile, {user.name}!"
```


Chaining Decorators

Multiple decorators can be applied to a function by stacking them.

```
def bold(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        return f"<b>{func(*args, **kwargs)}</b>"
    return wrapper

def italic(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        return f"<i>{func(*args, **kwargs)}</i>"
    return wrapper

@bold
@italic
def format_text(text):
    return text

print(format_text("Hello"))  # Output: <b><i>Hello</i></b>
```

Class Decorators

Decorators can also be applied to classes to modify their behavior.

```
def add_greeting(cls):
    cls.greet = lambda self: f"Hello, I'm {self.name}"
    return cls

@add_greeting
class Person:
    def __init__(self, name):
        self.name = name

p = Person("Alice")
print(p.greet())  # Output: Hello, I'm Alice
```


Class-Based Decorators

Decorators can be implemented as classes with __call__ method.

```
class CountCalls:
    def __init__(self, func):
        functools.update_wrapper(self, func)
        self.func = func
        self.num_calls = 0

    def __call__(self, *args, **kwargs):
        self.num_calls += 1
        print(f"Call {self.num_calls} of {self.func.__name__!r}")
        return self.func(*args, **kwargs)

@CountCalls
def say_hi():
    print("Hi!")

say_hi()  # Output: Call 1 of 'say_hi'
          #         Hi!
say_hi()  # Output: Call 2 of 'say_hi'
          #         Hi!
```

Built-in Decorators

Python includes several built-in decorators:

@property: converts a method into a property

@classmethod: creates a method that receives the class as first argument

@staticmethod: creates a method that doesn't receive implicit first argument

@abstractmethod: indicates abstract methods in abstract classes

Built-in Decorators

```
class Temperature:
    def __init__(self, celsius=0):
        self._celsius = celsius

    @property
    def celsius(self):
        return self._celsius

    @celsius.setter
    def celsius(self, value):
        self._celsius = value

    @property
    def fahrenheit(self):
        return self._celsius * 9/5 + 32

    @fahrenheit.setter
    def fahrenheit(self, value):
        self._celsius = (value - 32) * 5/9

temp = Temperature()
temp.celsius = 25
print(temp.fahrenheit)  # Output: 77.0
```

@property: converts a method into a property

The decorator marks the getter function.

To specify the setter use the automatically created decorator ***@function_name.setter***

You can also specify the behaviour on deletion with ***@function_name.deleter***

Built-in Decorators

@classmethod allows to declare methods that are related to the class (not the instances).

They can be useful to create alternative constructors.

Needed for the Singleton Pattern, useful for Factory methods.

```
class Person:
    species = "Homo sapiens"  # Class attribute

    def __init__(self, name, age):
        self.name = name
        self.age = age

    @classmethod
    def set_species(cls, new_species):
        cls.species = new_species  # Modifies the class attribute

    @classmethod
    def from_birth_year(cls, name, birth_year):
        current_year = 2025
        return cls(name, current_year - birth_year)  # Creates a new instance

# Using the class method to create an instance
p1 = Person.from_birth_year("Alice", 2000)
print(p1.name, p1.age)  # Output: Alice 25

# Changing the class attribute using a class method
Person.set_species("Homo technologicus")
print(Person.species)  # Output: Homo technologicus
```


Built-in Decorators

@staticmethod: defines a method that belongs to the class but does not access instance (self) or class (cls) attributes.

It behaves like a regular function but is grouped inside the class for organizational purposes.

```
class MathUtils:
    @staticmethod
    def add(a, b):
        return a + b

# Usage
result = MathUtils.add(3, 5)
print(result)  # Output: 8
```

Abstract classes

```python
from abc import ABC, abstractmethod

class Animal(ABC):
    @abstractmethod
    def make_sound(self):
        """Method that must be implemented by subclasses"""
        pass

class Dog(Animal):
    def make_sound(self):
        return "Woof!"

class Cat(Animal):
    def make_sound(self):
        return "Meow!"

# Trying to instantiate Animal directly will raise an error:
# animal = Animal()  # TypeError: Can't instantiate abstract class Animal with abstract method
make_sound

# But subclasses that implement all abstract methods can be instantiated:
dog = Dog()
cat = Cat()

print(dog.make_sound())  # Output: Woof!
print(cat.make_sound())  # Output: Meow!
```

@abstractmethod: defines abstract methods in an abstract base class (ABC).

A class that inherits from an ABC must implement all its abstract methods before it can be instantiated.

Decorator Best Practices

- Use *functools.wraps* to maintain the original function's metadata
- Keep decorators simple and focused on a single responsibility
- Document how your decorator modifies function behavior
- Consider potential performance impacts
- Use built-in decorators when appropriate instead of creating custom ones

Exercises

- Implement a *@deprecated* decorator that issues a warning when a deprecated function is called.
- Implement a *@retry(n)* decorator that retries a function up to n times if it raises an exception.
- Write a recursive function to calculate the n-element of the Fibonacci sequence (fib(n) = fib(n-1) + fib(n-2); fib(0) = 0; fib(1) = 1). Write a *@memorize* decorator that caches the output of the function. Show that the resulting implementation is faster than the naive implementation.

ITERATORS AND GENERATORS

Iterables and Iterators

An **iterable** is any object that can be looped over (lists, strings, dictionaries, etc.)

An **iterator** is an object that represents a stream of data, allowing you to access elements one at a time.

```
# Manual iteration using iterators
my_list = [1, 2, 3]

# Get iterator from the iterable
iterator = iter(my_list)

# Get elements one by one
print(next(iterator))  # Output: 1
print(next(iterator))  # Output: 2
print(next(iterator))  # Output: 3

# StopIteration exception is raised when no more items
try:
    print(next(iterator))
except StopIteration:
    print("No more elements!")
```


The Iterator Protocol

Objects that implement the iterator protocol must have:

__iter__() returns the iterator object itself

__next__() returns the next item or raises StopIteration when there are no more items

```
class CountUp:
    def __init__(self, start, stop):
        self.current = start
        self.stop = stop

    def __iter__(self):
        return self

    def __next__(self):
        if self.current > self.stop:
            raise StopIteration
        else:
            self.current += 1
            return self.current - 1

# Using the iterator
for num in CountUp(1, 5):
    print(num)  # Output: 1, 2, 3, 4, 5
```

Creating Custom Iterables

An iterable doesn't need to be an iterator itself; it only needs to return an iterator with its __iter__ method.

```
class EvenNumbers:
    def __init__(self, max_num):
        self.max_num = max_num

    def __iter__(self):
        return EvenNumbersIterator(self.max_num)

class EvenNumbersIterator:
    def __init__(self, max_num):
        self.current = 0
        self.max_num = max_num

    def __iter__(self):
        return self

    def __next__(self):
        if self.current > self.max_num:
            raise StopIteration
        else:
            self.current += 2
            return self.current - 2

# Using the iterable
for num in EvenNumbers(10):
    print(num)  # Output: 0, 2, 4, 6, 8, 10
```


Introduction to Generators

Generators provide an easy way to create iterators without implementing the full iterator protocol. They use the yield statement to return values one at a time.

```
# Generator function
def count_up(start, stop):
    current = start
    while current <= stop:
        yield current
        current += 1

# Using the generator
for num in count_up(1, 5):
    print(num)  # Output: 1, 2, 3, 4, 5

# Creating a list from a generator
numbers = list(count_up(1, 5))  # [1, 2, 3, 4, 5]
```

Generator Functions vs. Regular Functions

Generator functions:

- Use *yield* instead of *return*
- Return a generator object
- Maintain state between calls
- Execution is paused at each yield
- Memory efficient for large datasets

Generator Functions vs. Regular Functions

```python
# Regular function - returns all values at once
def get_squares_list(n):
    result = []
    for i in range(n):
        result.append(i * i)
    return result

# Generator function - yields one value at a time
def get_squares_generator(n):
    for i in range(n):
        yield i * i

# Compare memory usage for large values
squares_list = get_squares_list(1000000)  # Creates a list with 1 million items
squares_gen = get_squares_generator(1000000)  # Creates a generator object
```

Generator expressions

Generator expressions are similar to list comprehensions but return a generator object instead of a list.

```
# List comprehension (creates the entire list in memory)
squares_list = [x * x for x in range(10)]

# Generator expression (creates a generator object)
squares_gen = (x * x for x in range(10))

# List comprehension vs. generator expression for large data
import sys

list_comp = [x for x in range(10000)]
gen_exp = (x for x in range(10000))

print(f"List size: {sys.getsizeof(list_comp)} bytes")
print(f"Generator size: {sys.getsizeof(gen_exp)} bytes")
```

List size: 85176 bytes
Generator size: 192 bytes

State of generators

Generators maintain their state between calls and resume execution where they left off.

```
def counter():
    print("First yield")
    yield 1
    print("Second yield")
    yield 2
    print("Third yield")
    yield 3
    print("Done")

gen = counter()
print(next(gen))  # Prints "First yield" then 1
print(next(gen))  # Prints "Second yield" then 2
print(next(gen))  # Prints "Third yield" then 3

try:
    print(next(gen))  # Prints "Done" then raises StopIteration
except StopIteration:
    print("Generator exhausted")
```

Infinite Generators

Generators can represent infinite sequences without consuming infinite memory.

```
def fibonacci():
    a, b = 0, 1
    while True:
        yield a
        a, b = b, a + b

# Using the infinite generator
fib = fibonacci()
for _ in range(10):
    print(next(fib))  # First 10 Fibonacci numbers
```


Lazy Evaluation

Generators use lazy evaluation, calculating values only when needed.

```
def expensive_calculation(x):
    print(f"Computing for {x}...")
    # Simulate expensive operation
    import time
    time.sleep(0.5)
    return x * x

# Eager evaluation (list comprehension)
print("Creating list...")
squares_list = [expensive_calculation(x) for x in range(5)]
print("List created!")

# Lazy evaluation (generator expression)
print("Creating generator...")
squares_gen = (expensive_calculation(x) for x in range(5))
print("Generator created!")  # No calculations performed yet

print("Getting first value from generator...")
first_value = next(squares_gen)  # Only now is the first calculation performed
```

Chaining Generators

Generators can be composed and chained together to create data processing pipelines.

```
def generate_numbers(n):
    for i in range(n):
        yield i

def square(numbers):
    for number in numbers:
        yield number * number

def filter_even(numbers):
    for number in numbers:
        if number % 2 == 0:
            yield number

# Creating a pipeline
numbers = generate_numbers(10)  # 0, 1, 2, ..., 9
squared = square(numbers)       # 0, 1, 4, ..., 81
even_squares = filter_even(squared)  # 0, 4, 16, 36, 64

# Process the pipeline
for num in even_squares:
    print(num)
```


Yield From

The yield from statement (Python 3.3+) allows delegation to other generators.

```
def gen1():
    yield 1
    yield 2
    yield 3

def gen2():
    yield 4
    yield 5
    yield 6

def combined():
    yield from gen1()
    yield from gen2()

# Using the combined generator
for num in combined():
    print(num)  # Output: 1, 2, 3, 4, 5, 6
```

Recursive Generators

Generators can be recursively defined using *yield from*.

```
def flatten(nested_list):
    """Flatten a nested list structure."""
    for item in nested_list:
        if isinstance(item, list):
            yield from flatten(item)
        else:
            yield item

# Using the recursive generator
nested = [1, [2, [3, 4], 5], 6]
for num in flatten(nested):
    print(num)  # Output: 1, 2, 3, 4, 5, 6
```


Sending Values to Generators

Generators can also receive values using the *send()* method, allowing two-way communication.

```
def echo():
    while True:
        received = yield
        print(f"Received: {received}")

gen = echo()
next(gen)  # Prime the generator
gen.send("Hello")  # Output: Received: Hello
gen.send(42)       # Output: Received: 42
gen.close()        # Stop the generator
```

Coroutines with Generators

Generators can be used to create coroutines for more complex control flow:

```
def grep(pattern):
    print(f"Looking for {pattern}")
    while True:
        line = yield
        if pattern in line:
            print(line)

g = grep("python")
next(g)  # Prime the coroutine
g.send("python is great")       # Output: python is great
g.send("javascript is also cool")  # No output
g.send("python is better")      # Output: python is better
```


Coroutine Use Cases

Coroutines with generators are particularly useful when you need to manage cooperative multitasking, asynchronous processing, or streaming data:

1. Lazy Iteration Over Large Datasets
2. Pipelining Data Processing
3. Event-Driven and Asynchronous Programming
4. Managing State Between Function Calls
5. Simulating Concurrency Without Threads

Sending exceptions to generators

Generators have additional methods beyond next():

send(value) send a value into the generator

throw(exception) throw an exception inside the generator

close() stop the generator

```
def handler():
    try:
        while True:
            try:
                value = yield
                print(f"Handling: {value}")
            except ValueError:
                print("Caught ValueError!")
    finally:
        print("Generator closed")

h = handler()
next(h)           # Prime the generator
h.send("data")    # Output: Handling: data
h.throw(ValueError)  # Output: Caught ValueError!
h.close()         # Output: Generator closed
```


Sending exceptions to generators

Generators have additional methods beyond next():

send(value) send a value into the generator

throw(exception) throw an exception inside the generator

close() stop the generator

```
def handler():
    try:
        while True:
            try:
                value = yield
                print(f"Handling: {value}")
            except ValueError:
                print("Caught ValueError!")
    finally:
        print("Generator closed")

h = handler()
next(h)           # Prime the generator
h.send("data")    # Output: Handling: data
h.throw(ValueError)  # Output: Caught ValueError!
h.close()         # Output: Generator closed
```

Throws GeneratorExit Exception

Generators in the Standard Library

Python's standard library includes many generators:

```
import itertools

# Count indefinitely starting from 0
for i in itertools.count(0):
    if i > 5:
        break
    print(i)  # Output: 0, 1, 2, 3, 4, 5

# Cycle through elements
count = 0
for color in itertools.cycle(['red', 'green', 'blue']):
    if count >= 6:
        break
    print(color)
    count += 1  # Output: red, green, blue, red, green, blue

# Chain multiple iterables
for item in itertools.chain([1, 2], ['a', 'b']):
    print(item)  # Output: 1, 2, a, b
```


Exercises

- Create a generator function **fibonacci(n)** that yields the first n Fibonacci numbers.
- Write a generator function **prime_numbers(limit)** that yields all prime numbers up to a given limit.

CONTEXT MANAGERS

Context Managers

Context managers are a Python feature that allows you to allocate and release resources precisely when you want to.

The most common use is the *with* statement, which ensures that resources are properly managed regardless of whether an operation succeeds or fails.

```
# File handling without context manager
file = open('data.txt', 'r')
try:
    content = file.read()
finally:
    file.close()

# File handling with context manager (cleaner and safer)
with open('data.txt', 'r') as file:
    content = file.read()
# File is automatically closed when exiting the with block
```

Multiple Context Managers

You can use multiple context managers in a single *with* statement:

```
with open('input.txt', 'r') as input_file, open('output.txt', 'w') as output_file:
    data = input_file.read()
    output_file.write(data.upper())
```


How Context Managers Work

A context manager is any object that implements the context manager protocol, which consists of two methods:

__enter__() set up the context and return an object

__exit__() clean up resources when the context is exited

```
# Example of a custom context manager
class FileManager:
    def __init__(self, filename, mode):
        self.filename = filename
        self.mode = mode
        self.file = None

    def __enter__(self):
        self.file = open(self.filename, self.mode)
        return self.file

    def __exit__(self, exc_type, exc_val, exc_tb):
        if self.file:
            self.file.close()
        # Return False to propagate exceptions, True to suppress them
        return False

# Using our custom context manager
with FileManager('data.txt', 'r') as file:
    content = file.read()
```

Exception Handling in Context Managers

The __exit__ method receives information about any exception that occurred in the with block.

```
class DatabaseConnection:
    def __enter__(self):
        # Connect to a database
        print("Database connection opened")
        return self

    def __exit__(self, exc_type, exc_val, exc_tb):
        # Close the connection
        print("Database connection closed")

        # Log any exceptions
        if exc_type:
            print(f"Exception occurred: {exc_val}")

        # Return False to propagate exceptions, True to suppress them
        return False

with DatabaseConnection() as db:
    print("Working with database...")
    raise ValueError("Something went wrong")  # This will be propagated
```


Context Manager Example

```
# Example: A simple timer context manager
import time

class Timer:
    def __enter__(self):
        self.start = time.time()
        return self

    def __exit__(self, *args):
        self.end = time.time()
        self.elapsed = self.end - self.start
        print(f"Elapsed time: {self.elapsed:.2f} seconds")

# Usage
with Timer():
    # Do some time-consuming operation
    time.sleep(1.5)
```

The contextlib Module

Python's contextlib module provides utilities for working with context managers.

The @contextmanager decorator allows you to create a context manager using a generator function:

```
from contextlib import contextmanager

@contextmanager
def open_file(filename, mode):
    file = open(filename, mode)
    try:
        yield file  # This is what gets returned by the context manager
    finally:
        file.close()  # This is always executed, even if there's an exception

# Using our generator-based context manager
with open_file('data.txt', 'r') as file:
    content = file.read()
```


THE WALRUS OPERATOR

What is the Walrus Operator?

The walrus operator **:=** allows you to assign a value to a variable as part of an expression.

It was introduced in Python 3.8 (PEP 572) and is formally called an assignment expression.

Without the walrus operator:

```
n = len(my_list)
if n > 10:
    print(f"List is too long: {n}")
```

With the walrus operator:

```
if (n := len(my_list)) > 10:
    print(f"List is too long: {n}")
```

Key difference: the regular assignment **=** is a statement (cannot be used inside expressions). The walrus operator **:=** is an expression — it both assigns and returns the value.

The name "walrus" comes from the := symbol resembling a walrus face rotated 90°

Use Case: while loops

The walrus operator shines in while loops where you need to read input and check a condition simultaneously.

Traditional approach:

```
# Read and process until empty
line = input("Enter text: ")
while line != "quit":
    print(f"You said: {line}")
    line = input("Enter text: ")
```

Problem: input() is called twice (DRY violation)

With walrus operator:

```
# Read and process until empty
while (line := input("Enter: ")) != "quit":
    print(f"You said: {line}")
```

Benefit: single call, cleaner loop

Key difference: the regular assignment **=** is a statement (cannot be used inside expressions). The walrus operator **:=** both assigns and returns the value.

The name "walrus" comes from the := symbol resembling a walrus face rotated 90°

Use Case: comprehensions & filtering

The walrus operator avoids calling an expensive function twice inside a comprehension.

Without (redundant computation):

```
results = [compute(x) for x in data if compute(x) > threshold]
```

Problem: compute(x) is called TWICE per element — once to filter, once to keep

With walrus operator:

```
results = [y for x in data if (y := compute(x)) > threshold]
```

Benefit: compute(x) called once, result reused as y

Use Case: conditional logic & chaining

Useful when you need to compute, test, and use a value in an if/elif chain.

```
# Avoids recomputing or introducing
# temp variables before the if
if (n := len(items)) == 0:
    print("Empty")
elif n == 1:
    print("Single item")
else:
    print(f"Multiple items: {n}")

# Common pattern with dict.get()
env = os.environ
if (db := env.get("DATABASE_URL")):
    connect(db)
elif (db := env.get("DB_FALLBACK")):
    connect(db)
else:
    raise RuntimeError("No DB")
```

Scope note: unlike regular comprehension variables, a walrus-assigned variable **leaks into the enclosing scope**. This is by design (PEP 572), but be aware of it.

Best Practices & Pitfalls

DO use when…

- It eliminates redundant computation
- It reduces DRY violations in loops
- It simplifies while-loop patterns
- The scope of the variable is clear

DON'T use when…

- A simple assignment would be clearer
- It makes a line excessively complex
- It harms readability for no real gain
- Nested walrus operators (never do this!)

Best Practices & Pitfalls

Anti-pattern:

```
# Too clever. Hard to read
result = [(y := f(x), x/y)
          for x in range(1, 5)]
```

Prefer clarity:

```
# Clear and readable
result = []
for x in range(1, 5):
    y = f(x)
    result.append((y, x/y))
```

Exercises

- Rewrite the following code using the walrus operator:

```
values = [4, 8, 15, 16, 23, 42]
filtered = []
for v in values:
    doubled = v * 2
    if doubled > 20:
        filtered.append(doubled)
```

- Write a while loop using := that reads integers from input() and computes their running sum. Stop when the user enters 0. Print the final sum.
- Given a list of strings, use a list comprehension with := to collect only those strings whose length (as computed by len()) exceeds 5, storing the lengths alongside the strings as tuples: [(string, length), ...]

FINAL PYTHON EXERCISE

The 8-queens problem

A classic problem in computer science and mathematics

First proposed by chess player Max Bezzel in 1848

Goal: Place 8 queens on a standard 8×8 chessboard so that no queen threatens any other queen

A queen can attack any piece in the same row, column, or diagonal

The Rules of Chess for Queens

In chess, the queen can move:

- Horizontally (along its row)
- Vertically (along its column)
- Diagonally (in all four diagonal directions)

For queens to not threaten each other, they must not share:

- The same row
- The same column
- The same diagonal

Algebraic notation

Every position on a standard chess board has a coordinate

8	a8	b8	c8	d8	e8	f8	g8	h8
7	a7	b7	c7	d7	e7	f7	g7	h7
6	a6	b6	c6	d6	e6	f6	g6	h6
5	a5	b5	c5	d5	e5	f5	g5	h5
4	a4	b4	c4	d4	e4	f4	g4	h4
3	a3	b3	c3	d3	e3	f3	g3	h3
2	a2	b2	c2	d2	e2	f2	g2	h2
1	a1	b1	c1	d1	e1	f1	g1	h1
	a	b	c	d	e	f	g	h

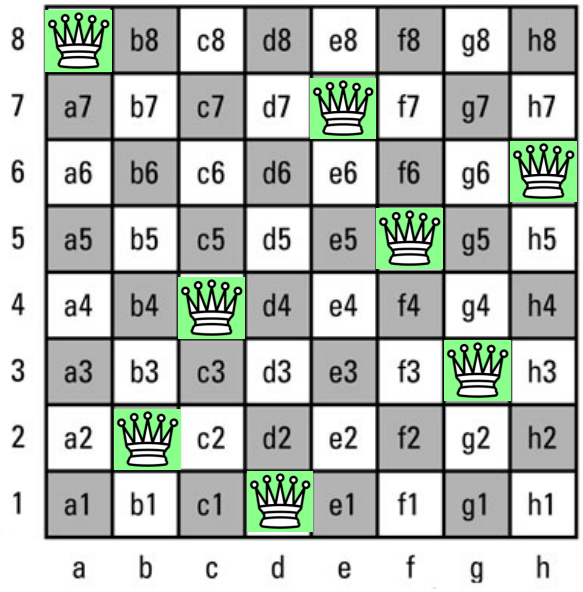
Visual Representation

Examples of an invalid placement (queens threatening each other):

8	a8	b8	c8	d8	e8	f8	g8	h8
7	a7	♕	c7	d7	e7	f7	g7	h7
6	a6	b6	c6	d6	e6	f6	g6	h6
5	a5	b5	c5	d5	e5	f5	g5	h5
4	a4	b4	c4	d4	e4	f4	g4	h4
3	a3	b3	c3	d3	e3	♕	g3	h3
2	a2	b2	c2	d2	e2	f2	g2	h2
1	a1	b1	c1	d1	e1	f1	g1	h1
	a	b	c	d	e	f	g	h

8	a8	b8	c8	d8	e8	f8	g8	h8
7	a7	b7	c7	♕	e7	f7	g7	h7
6	a6	b6	c6	d6	e6	f6	g6	h6
5	a5	b5	c5	d5	e5	f5	g5	h5
4	a4	b4	c4	d4	e4	f4	g4	h4
3	a3	b3	c3	♕	e3	f3	g3	h3
2	a2	b2	c2	d2	e2	f2	g2	h2
1	a1	b1	c1	d1	e1	f1	g1	h1
	a	b	c	d	e	f	g	h

Example of a valid solution



Why Is This an Interesting Problem?

Combinatorial Explosion: for an 8×8 board, there are 64 possible positions for the first queen, 63 for the second, etc.

Without constraints: 64!/(64-8)! = 178,462,987,637,760 possible arrangements

But only 92 solutions (12 unique solutions accounting for symmetry)

Constraint Satisfaction: exemplifies constraint problems in computer science

Algorithmic Thinking: requires development of efficient search strategies

Approaches to solving the problem

Brute Force: try all possible combinations (extremely inefficient)

Backtracking: place queens one by one, backtrack when a conflict is detected. Most common and efficient approach for this problem.

Genetic Algorithms: evolve solutions through simulated natural selection.

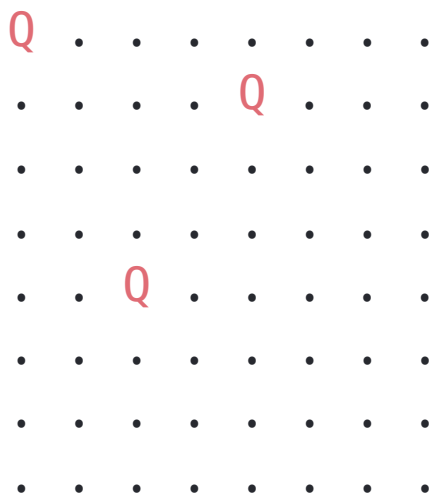
Constraint Satisfaction Techniques: formulate as a constraint satisfaction problem.

Representation in python

How would you represent the board and queens in Python?

Visual representation

You can use a visual representation like this:



Backtracking Algorithm Concept

1. Start in the leftmost column
2. If all queens are placed, return true
3. Try placing a queen in all rows of the current column
    1. If queen can be placed safely, mark it as placed
    2. Recursively check if placing queen here leads to a solution
    3. If placing queen leads to a solution, return true
    4. If placing queen doesn't lead to a solution, backtrack (remove queen) and try other rows
4. If all rows have been tried and no solution is found, return false

Challenge

Implement the 8-queens problem solver using the backtracking approach

Test the solution with different board sizes (4×4, 6×6, etc.)

Visualize the solution(s) in the console

Analyze the time complexity and the efficiency of your code
Can you optimize further?

Challenge

Implement the 8-queens problem solver using the backtracking approach

Test the solution with different board sizes (4×4, 6×6, etc.)

Visualize the solution(s) in the console

Analyze the time complexity and the efficiency of your code
Can you optimize further?

-> **Leaderboard:** at the end we will measure the timing of your solution for 4,6,8, … 32 board sizes and build a leaderboard with a winner.